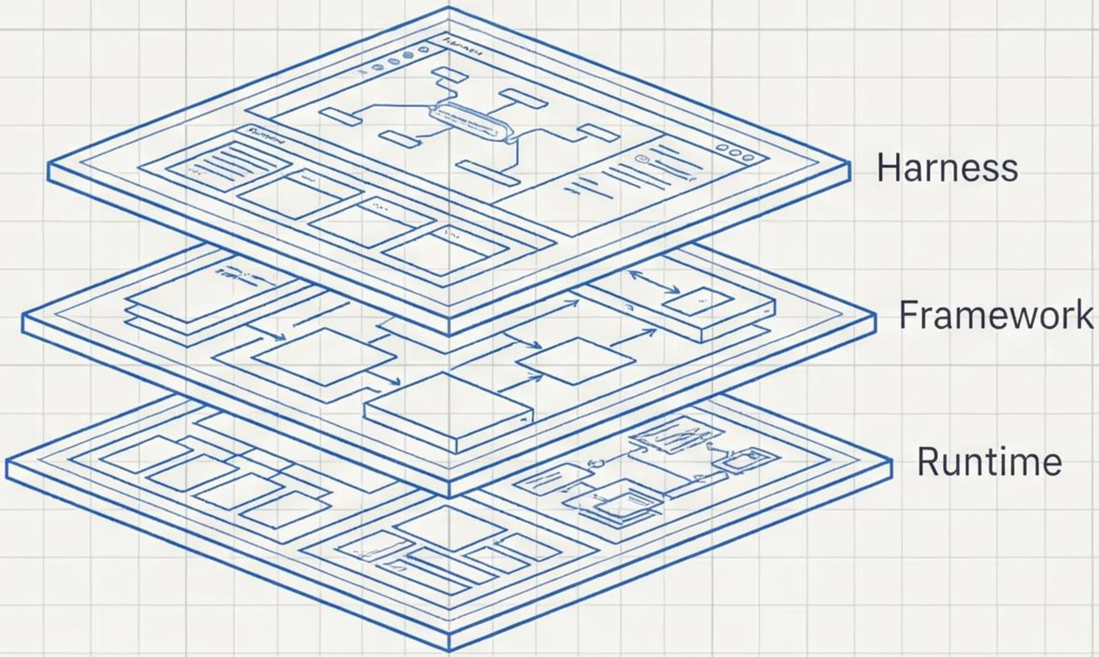
Deep Agents 实战

第 1 讲：Agent 开发的三个层次



构建 AI 编程助手的困境

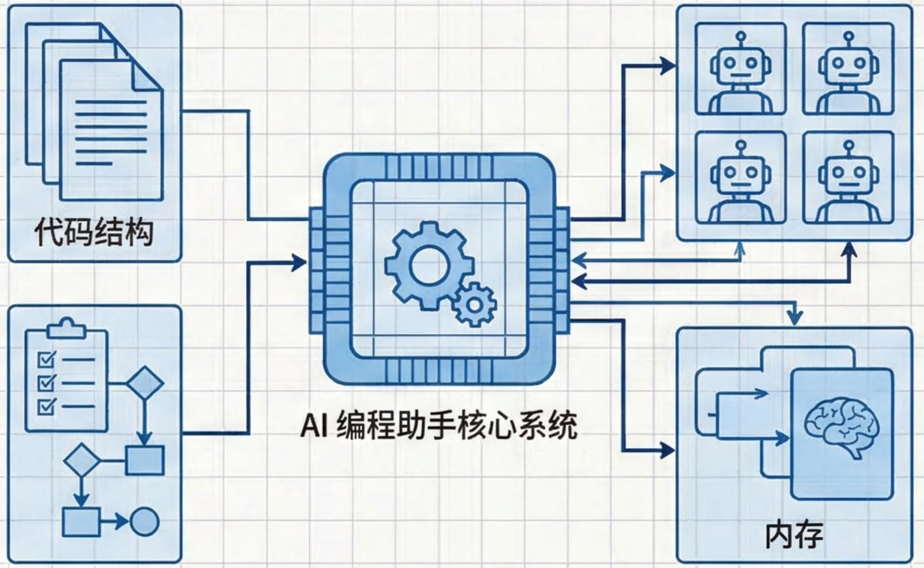
你需要的能力远比你想的多

困境（The Dilemma）



- 读取源代码文件、理解代码结构
- 制定修改计划、分步执行、追踪进度
- 委派子任务给专门的 Agent
- 跨对话记住用户偏好
- 每个"认真"的 Agent 都需要这些 → **重复造轮子**

解决方案（The Solution）



统一的集成化系统：
能力模块化、可复用、无需重复建设

Agent 开发的三个层次

自底向上构建，不是互相替代

Harness · Deep Agents

装配文件工具、上下文管理与任务委派

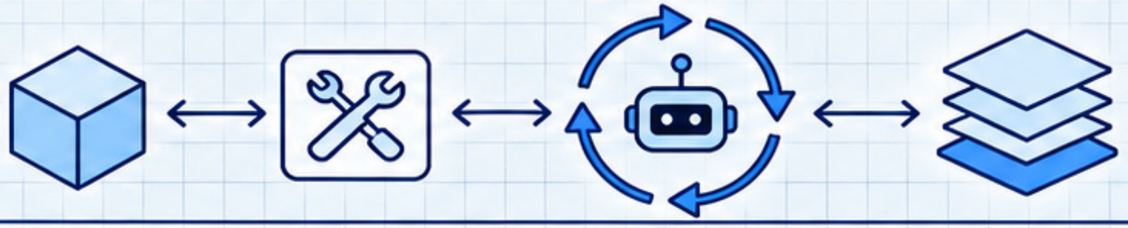
按需配置规划、Skills 与记忆



构建于

Framework · LangChain

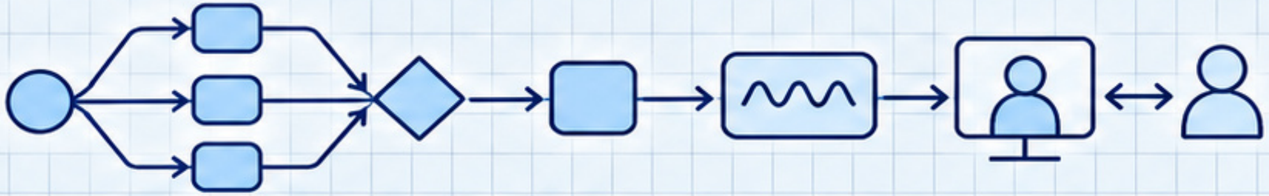
模型抽象 · 工具接口 · Agent 循环 · Middleware



构建于

Runtime · LangGraph

图执行 · 状态管理 · 流式输出 · 人机协作



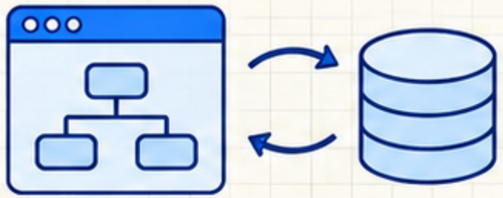
选择抽象层，取决于你需要多少预置能力与执行控制。

底层：LangGraph（Runtime）

Agent 的执行基础设施

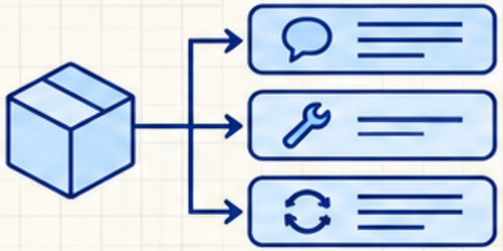
持久化执行

配置 Checkpointer 后
保存与恢复执行状态



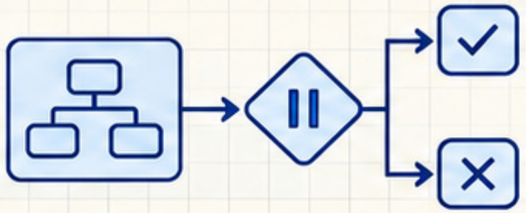
流式输出

输出模型内容、工具事件
与状态更新



人机协作

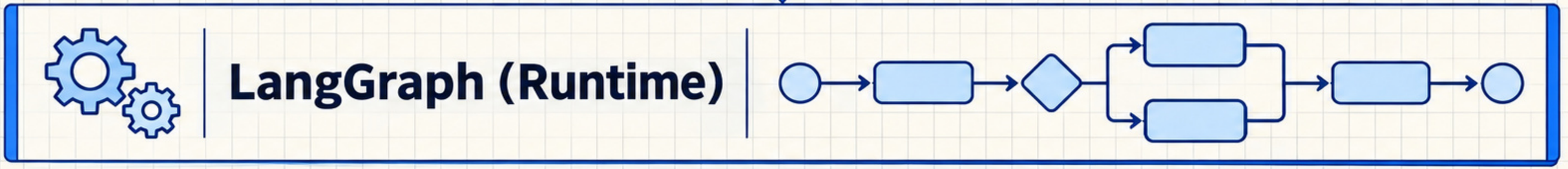
在配置的节点或工具
调用前暂停审批



状态管理

同一 thread_id 续接线程；
跨重启需持久化存储





⚠ 恢复能力依赖正确配置；副作用仍需考虑幂等性。流式事件不等于模型隐藏思维。

中间层：LangChain（Framework）

在 LangGraph 之上，标准化模型与工具循环

模型抽象

统一调用接口



工具接口

注册业务动作



Agent 循环

模型 → 工具 → 模型



Middleware

在执行环节扩展行为

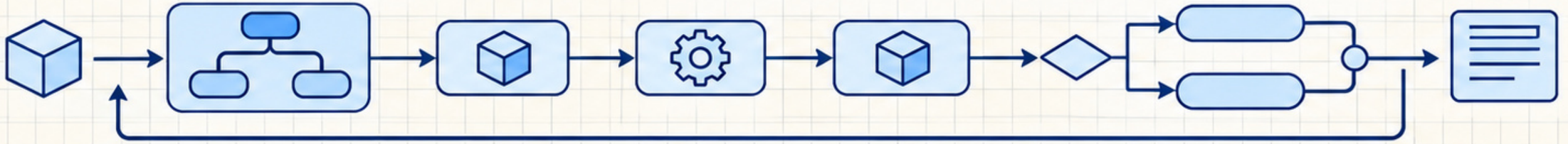


API 形态示意

```
from langchain.agents import create_agent

agent = create_agent(
   model="gpt-4.1",
   tools=[web_search, calculator],
   system_prompt="You are a helpful assistant."
)
```

LangGraph Runtime

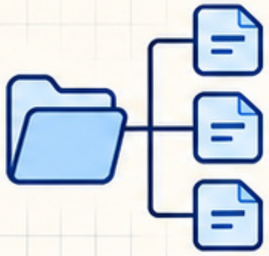


web_search 和 calculator 由应用自行实现并注册；这不是独立可运行示例。

上层：Deep Agents（Harness）

预置核心能力，按需开启策略 · v0.7

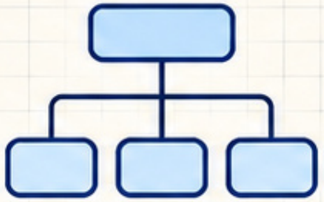
文件工具



read_file · write_file · edit_file
delete · ls · glob · grep

七个文件操作工具

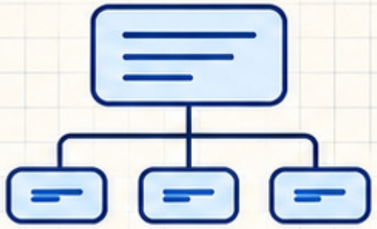
任务规划（按需）



TodoListMiddleware

显式启用后提供 write_todos

子任务委派



task

独立上下文处理子任务，返回结果



记忆（按需）

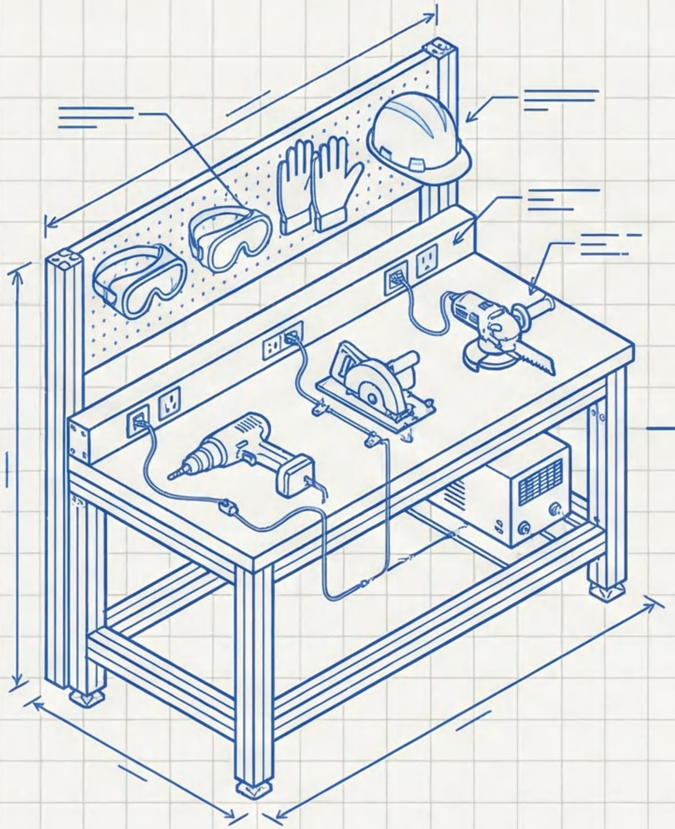


memory= 加载指定记忆文件

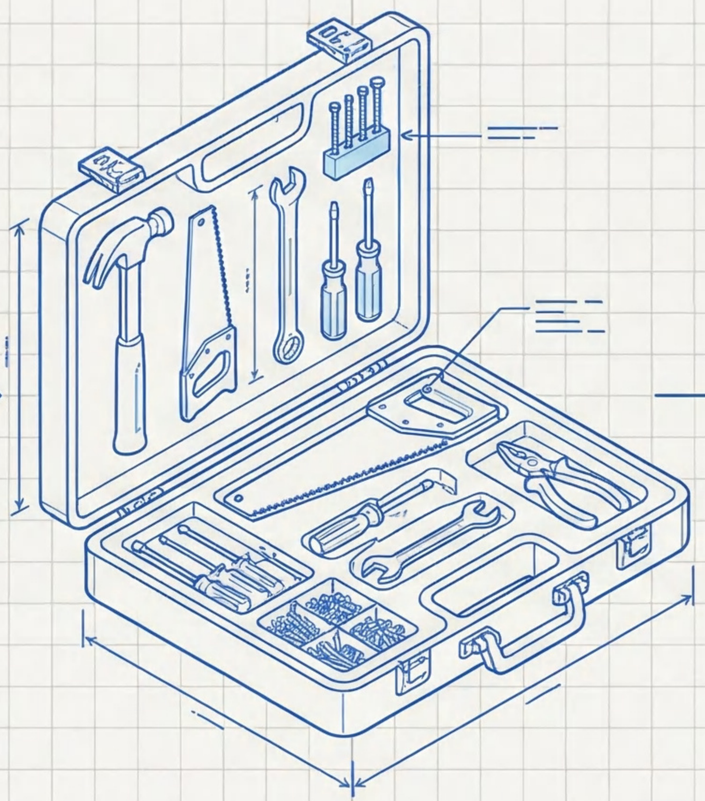
跨线程保存取决于持久化 Backend

核心还包括摘要与大结果卸载；具体工具受 Backend、权限及 Harness profile 约束。

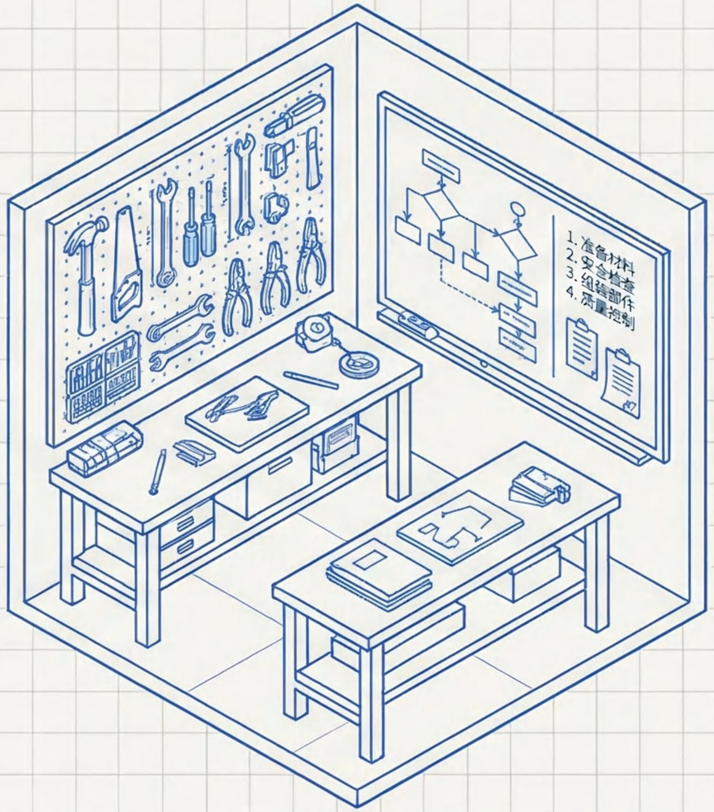
三层架构的直觉类比



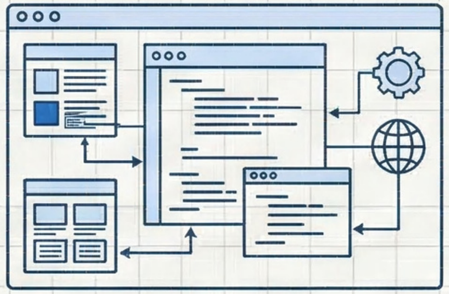
Runtime = 工作台、电源、安全护具（底层基础设施）



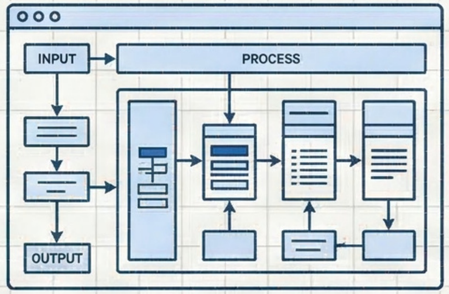
Framework = 锤子、锯子、钉子（标准化工具）



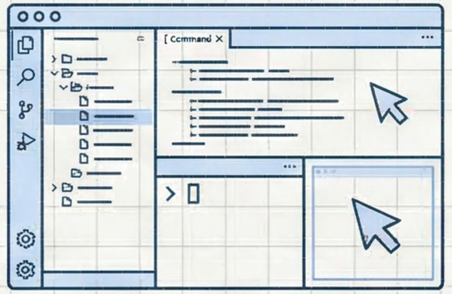
Harness = 装好的工具间，工具挂墙上，流程贴白板（开箱即用）



Claude Code



Manus



Cursor

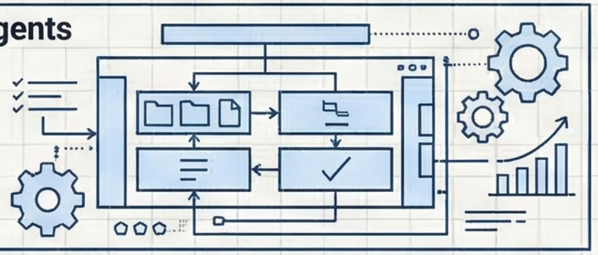
Claude Code、Manus、Cursor 的共性：

- ✓ 文件系统操作
- ✓ 子任务委派
- ✓ 任务规划能力
- ✓ 上下文管理策略

为什么需要 Harness?

成功的 Agent 产品都长得差不多

Harness 的价值：
把验证过的模式固化下来



Deep Agents 技术全景

装配 Middleware → 构建 Agent 循环 → 编译执行图

应用配置

model · tools · system_prompt · backend



create_deep_agent()

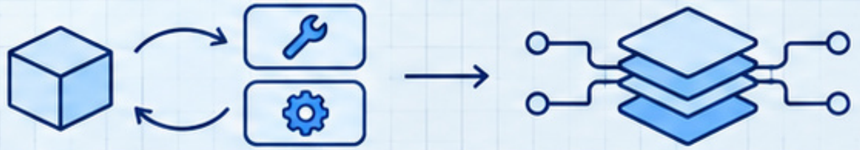
核心：文件系统 · 子 Agent · 摘要

按需：Todo · Skills · memory · 人工审批



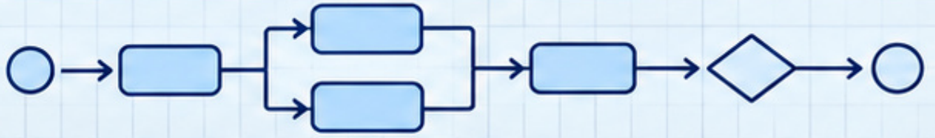
LangChain create_agent()

模型—工具循环 + Middleware hooks



LangGraph

编译执行图 · invoke() / stream()



LangSmith

追踪 · 调试 · 评测





Tool：模型请求的动作

Middleware：提供能力与执行拦截

Backend：文件存储与环境操作

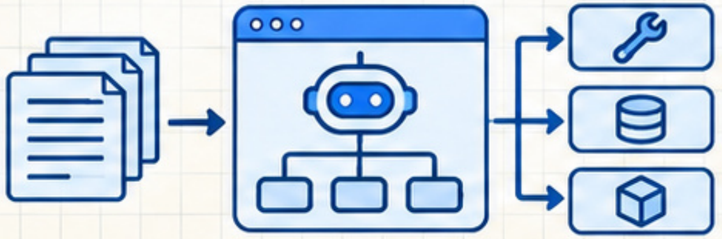
Checkpointer 保存线程执行状态；**Store** 可保存跨线程数据。持久化均需相应配置。

三层如何选择与组合

从需求出发，不把自主性与固定工作流对立起来

Deep Agents

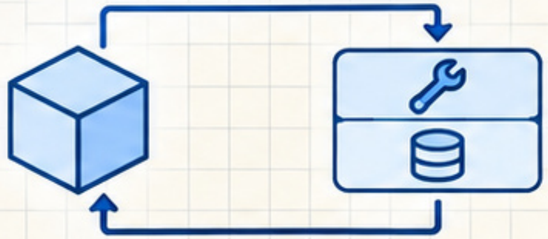
`create_deep_agent()`



需要文件、上下文管理与委派
配置业务工具与所需能力

LangChain

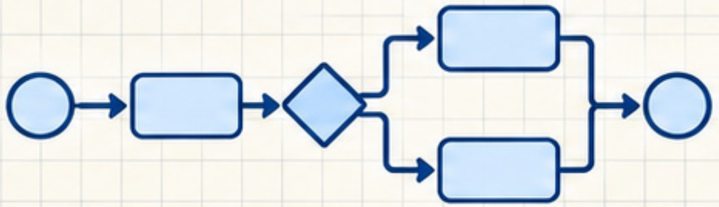
`create_agent()`



需要标准模型—工具循环
自行组合中间件与上下文策略

LangGraph

`StateGraph`



需要精确控制节点与路由
编排确定性步骤和自主 Agent

可在 LangGraph 中组合使用



输入校验



Deep Agent 研究



人工审核

也可把编译后的工作流包装为 CompiledSubAgent（见第5章）。

本讲回顾



- Runtime → Framework → Harness，自底向上



- Harness = 验证过的 Agent 模式的固化



- Deep Agents 的核心：Context Engineering



- 下一讲：Context Engineering 与竞品对比

下一讲预览：Context Engineering 与竞品对比

